

Deliverable report

“Alessio Blascovich”: Mat: 257561, alessio.blascovich@studenti.unitn.it, GitRepo:
<https://github.com/elblasco/GPU-Computing-2025-257561>

Abstract—The sparse matrix-dense vector multiplication (SpMV) is an algebraic operation in which a sparse matrix is multiplied by a dense vector. SpMV is widely used in real-world applications such as scientific and engineering computing, data mining, and graph analysis.

This deliverable discusses the implementation and the performance analysis of a series of SpMV algorithms for COO matrices. These algorithms are implemented for a single-core CPU and a CUDA capable GPU. Regarding the GPU implementations, no threads/warp level synchronisation has been used, only atomic operations to keep the resulting vector consistent.

Index Terms—Sparse Matrix, SpMV, CUDA, Parallelization, Storage Format

I. INTRODUCTION

SpMV is considered a fundamental algebraic operation in many scientific fields such as computational fluid dynamics, optimisation and graph algorithms.

The main challenges in implementing an efficient SpMV algorithm differ in CPU and GPU, and are strictly bound to the type of matrix representation used (COO, CSR, ELL,...). For the purpose of this deliverable only COO representation is taken into account.

From the CPU point of view, the main issue is to maximise the cache hits throughout the execution, data consistency is not a problem since only single core CPUs were used.

On the GPU side the problem of maximising cache hits remains, and on top of that, race conditions are added. Since in COO format the length of a row is not specified, multiple threads could be reading different portions of the same row and therefore, writing on the same output memory area.

II. PROBLEM STATEMENT

The SpMV operation is formally defined in Equation 1, where A is a $n \times m$ sparse matrix, x a $m \times 1$ dense vector and y another $n \times 1$ dense vector.

$$y = Ax \quad (1)$$

A. Storage Format

The coordinate (COO) format is a representation of sparse matrix. COO uses three different arrays to store all the necessary information. One array dedicated to store all the non-zero values, while the other two contain the corresponding row and column indices. For the sake of simplicity, all the input matrices are assumed to be sorted in row-major order, this is done by transposing all the input matrices which were sorted in column-major order.

B. Parallelization

Two different kernels have been used for the CUDA capable GPU, although in both of them the workload is divided equally among all the threads used. The main difference is the pattern used to access the global memory from a thread. In the first approach each thread jumps of stride equivalent to the number of threads used, while in the second approach a thread access to a continuous area of memory long as the total number of non-zero element in the matrix divided by the number of threads used.

III. STATE OF THE ART

Starting from the implementations presented in this deliverable, performances could be drastically enhanced. Fukaya et al. [1], despite focussing on the HDC storage format, improved the SpMV CPU based by exploiting diagonalization patterns, SIMD instructions and *OpenMP* directives. On the GPU side Chu et al. [2] used the CSR format and a more sophisticated load balancing to improve the performances. NVIDIA itself made available a set of libraries to manage and use COO formatted matrices, namely *cuSPARSE*.

IV. METHODOLOGY AND CONTRIBUTIONS

The tests has been carried out performing multiple rounds of caches warm up and then executing the various algorithm after. The data have been collected throughout the execution since only for the first warm up cycle there were substantially data variations.

A. CPU implementation

The CPU simply loop through all the elements and performs the operations required for the matrix multiplication element-wise. The CPU Algorithm 1 maximise the cache hit since it reads, sequentially, elements that are stored in a contiguous memory area, this holds for `res[row[i]]` and `val[i]`.

Algorithm 1 SpMV algorithm on CPU

Require: The input vectors *val*, *row* and *col* of size *nnz*, array *arr* of size *columns* and an array *res* of size *rows*.

```
1: procedure FUNCTION(row, col, val, arr, res, ...)  
2:   while i in {0...nnz} do  
3:     res[row[i]] = res[row[i]] + (val[i] · arr[col[i]])  
4:     i = i + 1  
5:   end while  
6: end procedure
```

In particular instance I had to use my personal computer since the cluster version of *valgrind* has some trouble with

the cache profile tool. Due to hardware limitation I performed the cache profiling using smaller matrices. To have clearer I did not perform any warm-up cycle sine I want to measure the effectiveness of my algorithm on “pseudo-empty” caches.

Matrix	D1 r miss	D1 w miss
lp_nug05	1.5%	.5%
rdb2501	.8%	.3%
fpga_dcop_15	.3%	.2%

TABLE I
L1 CACHE MISSES

Matrix	LLd r miss	LLd w miss
lp_nug05	.7%	.3%
rdb2501	.4%	.2%
fpga_dcop_15	.1%	.1%

TABLE II
L2 AND L3 CACHE MISSES

B. GPU

For the GPU implementation I have created two kernel, they differ on the access pattern used for the memory. In Algorithm 2 every thread jumps to the next element which has index $current\ index + numThreads$. In Algorithm 3 each thread accesses $portion$ elements which are stored in a contiguous area of memory.

Algorithm 2 SpMV algorithm with striding

Require: The input vectors val , row and col of size nnz , array arr of size $columns$, an array res of size $rows$ and a number $portion$.

```

1: procedure FUNCTION( $row, col, val, arr, res, \dots$ )
2:    $start \leftarrow threadId$ 
3:    $end \leftarrow (portion \cdot numThreads) + start$ 
4:   while  $i$  in  $\{start \dots \min\{end, nnz\}\}$  do
5:      $atomicAdd(\&res[row[i]], val[i] \cdot arr[col[i]])$ 
6:      $i = i + numThreads$ 
7:   end while
8: end procedure
```

Algorithm 3 SpMV algorithm with contiguous chunk

Require: The input vectors val , row and col of size nnz , array arr of size $columns$, an array res of size $rows$ and a number $portion$.

```

1: procedure FUNCTION( $row, col, val, arr, res, \dots$ )
2:    $start \leftarrow portio \cdot threadId$ 
3:    $end \leftarrow portion + start$ 
4:   while  $i$  in  $\{start \dots \min\{end, nnz\}\}$  do
5:      $atomicAdd(\&res[row[i]], val[i] \cdot arr[col[i]])$ 
6:      $i = i + 1$ 
7:   end while
8: end procedure
```

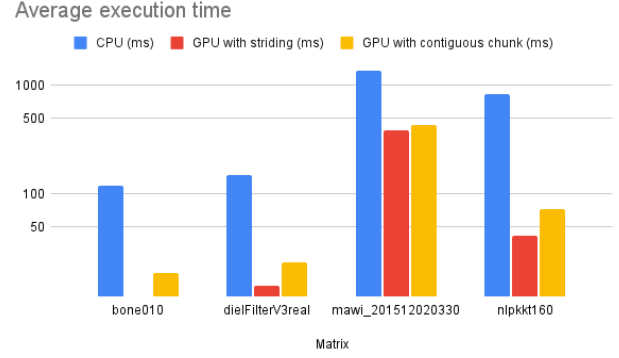


Fig. 1. Average Execution Time

As shown in Figure 1 the fastest algorithm is algorithm 2. This follows from the fact that every thread within the same SM (Streaming Multiprocessor) access elements that are spacially adjacent in the memory. Contrary, Algorithm 3 has a higher rate of cache miss since, threads in the same SM may need to access elements far away from each other. All the other results are shown and described in Section VI.

V. SYSTEM DESCRIPTION AND EXPERIMENTAL SET-UP

A. System Description

Table III shows the various software used in the deliverable, excluded the building tools. It is worth mentioning again that, for *valgrind*, I had to do a rollback on my pc since the last version (same of the cluster) has some problem with the tool *cachegrind*.

System	GCC	Valgrind	CUDA
Cluster	11.5.0	3.23.0	12.5.0
PC	15.1.1	3.20.0	12.8.1

TABLE III
SYSTEMS SOFTWARES

System	Processor	Cores per Socket	RAM	Accelerator
Cluster	Intel(R) Xeon(R) Silver 4214	48 at 2.2 GHz	400 GB	NVidia A30
PC	AMD Ryzen 5 3500U	8 at 2.1 GHz	8 GB	Radeon Vega Mobile Gfx

TABLE IV
SYSTEMS DETAILS

B. Dataset description

To add a bit more of generality to the dataset I picked four matrix from four different kinds from the site <https://sparse.tamu.edu/>.

- 1) Model Reduction Problem;
- 2) Electromagnetics Problem;
- 3) Undirected Weighted Graph;
- 4) Optimization Problem.

Name	Pattern Entries	Symmetric	Kind
bone010	71,666,325	Yes	1
dielFilterV3real	89,306,020	Yes	2
mawi_201512020330	480,047,894	Yes	3
nlpkkt160	229,518,112	Yes	4

TABLE V
INPUT MATRICES

C. Experimental Set-up

Explain the benchmark metrics and all the experimental set-up (warm-up cycles, number of performed runs...).

VI. EXPERIMENTAL RESULTS

Present and discuss results. Include plots and tables when required (like Figure ??). Do not simply describe the figures; criticise the achieved results by underlining how they confirm/differ from the expected outcomes described in Section IV.

VII. CONCLUSIONS

[max 200 words]

Summarize findings and future work ...

REFERENCES

- [1] T. Fukaya, K. Ishida, A. Miura, T. Iwashita, and H. Nakashima, "Accelerating the spmv kernel on standard cpus by exploiting the partially diagonal structures," 2021. [Online]. Available: <https://arxiv.org/abs/2105.04937>
- [2] G. Chu, Y. He, L. Dong, Z. Ding, D. Chen, H. Bai, X. Wang, and C. Hu, "Efficient algorithm design of optimizing spmv on gpu," in *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing*, ser. HPDC '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 115–128. [Online]. Available: <https://doi.org/10.1145/3588195.3593002>